Program IPO & Requirements Decomposition

Engineer Samantha Chorm

Section #1 - Program IPO (Input, Processing,Output)

ProgramTask: *The program will read an inputted file name and display each line with the line numbers followed by a colon. It should print 24 lines at a time.*

ProgramInput: *1 file input to read, 1 user input for every 24 lines displayed*

ProgramOutput: *The program will output line numbers right-aligned. All lines will be displayed with their line number and colon on the left before returning 0.*

Section #2 - Design Considerations

Goal: To get a file name from user and display all lines of that file with line numbers.

(INPUT) user enters string userFile
(FUNCTION) read userFile by storing string into ifstream fileName
(OUTPUT) display each line as "#: line", with # counting up from 1
If the file is longer than 24 lines, require user input to press ENTER until every line is displayed
END

Section#3 - Order of operations/flow of control

```
/*
OUTPUT "Enter the file name: "
INPUT userFile

CALL FUNCTION open_file(userFile, inFile)

WHILE (!inFile.is_open()) {
        OUTPUT "ERROR: File not found"
        OUTPUT "Enter the file name: "
        INPUT userFile
        CALL open_file(userFile, inFile)
}

CALL FUNCTION display_file(inFile)

END return 0
*/
```

Section #4 - Program Requirements

Needs 2 INPUTS from user (1 STRING for getting file name and 1 IFSTREAM for opening and reading the actual file), AND 1 FUNCTION (1 VOID to open, read, and display).

LIBRARIES NEEDED: <iostream>, <fstream>, <string>, <iomanip>

2 INPUTS - 1 input to get STRING userFile to open as IFSTREAM inFile (possible looping if user doesn't enter a valid file name).1 input "ENTER" to continue displaying lines.
STRING stores file name according to what user enters.
IFSTREAM is initialized from that input. It reads the actual file.

1 FUNCTION - 1 VOID (display_file) opens the reads the file found from inFile, then display the result, or what it's reading from that file.
VOID display_file should use ifstream& file.
While displaying, the function's code should include a WHILE loop for keeping track of STRING line.
Also use INT lineCount = 0 for listing # of lines.
It should look like this: WHILE (getline(input, line)): lineCount++, cout << lineNum << ": " << line << endl...

INT lineCount is used to determine if a .txt file is over/under 24 lines. If it has exactly 24 lines, it will only have to display once.
IF (lineCount % 24 == 0), THEN cout << "\tPress ENTER to continue...", cin.get()

OPTIONAL FUNCTION - 1 VOID (open_file) to validate whether the file opened or not. It should use const string& input and ifstream& file.
First add file.open(input), then add IF (file.is_open()), THEN proceed. Can print "File opened successfully."
ELSE IF (file.fail()), THEN say "File failed to open." Reprompt user to enter the file name again.

FORMAT: Every line printed should be aligned with setw(3).

Section #5 – Functions (required for each function in the program)

Function IPO (Input, Processing, Output): complete an IPO section **for each function** in the program.

Function name: *open_file*

Declared as: *void, parameters (const string& input, ifstream& file)*

Task: *Opens and validates if a user-inputted file name has opened or not.*

External requirements: *#include <fstream>, #include <string>*

Internal requirements: *Local stream state checks*

Internal operation: *Checks if file.is_open() or file.fail()*

Return Value: *None*

Function name: *display_file*

Declared as: *void, parameter (ifstream& file)*

Task: *Reads opened file by line and displays content with right-aligned line numbers, pausing every 24 lines.*

External requirements: *#include <iostream>, #include <fstream>, #include <string>, #include <iomanip>*

Internal requirements: *string line, int lineCount = 0*

Internal operation: *Loops using getline(file, line), lineCount++, prints setw(3) << lineCount << ": " << line, and prompts for ENTER key with cin.get() if (lineCount % 24 == 0).*

Return Value: *None*